

Sistemas Operativos

Facultad de Informática
Universidad Complutense de Madrid

Práctica 4: Procesos e Hilos



Outline

1 Objetivos

2 Desarrollo

Objetivos de la Práctica y contexto general

- **Afianzar el uso de la API POSIX** para la gestión de procesos e hilos en C.
- **Comprender el manejo de ficheros** cuando se accede a ellos de forma concurrente.
- **Analizar las diferencias esenciales** entre el aislamiento de procesos y el espacio compartido de los hilos.
- **Experimentar con señales** del sistema y control de temporizadores.



Estructura del Trabajo



- Crear un directorio común para la práctica.
- Crear un subdirectorio independiente para cada ejercicio (`ejercicio1`, `ejercicio2`, etc.).
- Utilizar los archivos de partida y los *Makefiles* del archivo `ficheros_p4.tar.gz`.

Ejercicio 1: Creación de Procesos (run_commands)

- **Objetivo:** Desarrollar un programa que ejecute comandos indicados por el usuario mediante procesos hijos.
- **Uso de `getopt()`:** El programa debe procesar tres opciones principales.

Opciones a implementar:

- `-x <comando>`: Ejecuta un único comando en un proceso hijo y espera su terminación.
- `-s <fichero>`: Lee un archivo línea por línea y ejecuta los comandos de forma secuencial.

Ejemplo de ejecución

Terminal #1

```
# Testing -x switch
$ ./run_commands -x ls
Makefile        run_commands    run_commands.c  run_commands.o  test1           test2

$ ./run_commands -x "echo hello"
Hello

# Testing -s switch
$ ./run_commands -s test1
@@ Running command #0: echo hello
hello
@@ Command #0 terminated (pid: 1439, status: 0)

@@ Running command #1: sleep 2
@@ Command #1 terminated (pid: 1440, status: 0)
@@ Running command #2: ls -l
total 88
-rw-r--r--@ 1 usuarioso  usuarioso   267 Oct 20 11:34 Makefile
-rwxr-xr-x  1 usuarioso  usuarioso  9960 Oct 20 11:58 run_commands
-rw-r--r--  1 usuarioso  usuarioso  4332 Oct 20 11:57 run_commands.c
-rw-r--r--  1 usuarioso  usuarioso  8984 Oct 20 11:58 run_commands.o
-rw-r--r--@ 1 usuarioso  usuarioso  31 Oct 20 11:34 test1
-rw-r--r--@ 1 usuarioso  usuarioso  41 Oct 20 11:46 test2
@@ Command #2 terminated (pid: 1443, status: 0)
@@ Running command #3: false
@@ Command #3 terminated (pid: 1444, status: 256)
```

Ejercicio 1: Detalles Técnicos y Preguntas

- Se proporciona la función `parse_command()` para trocear las cadenas en un formato válido para `argv`.
- Se debe implementar la función con el prototipo: `pid_t launch_command(char** argv);`

Preguntas a responder tras el desarrollo:

1. ¿Qué ocurre si el argumento de `-x` no se pasa entrecomillado? ¿Hay diferencia entre comillas simples y dobles? (Ejemplo: `echo $HOME`).
2. ¿Es posible utilizar `execlp()` en lugar de `execvp()`? ¿Qué limitaciones tendría?
3. ¿Qué ocurre al ejecutar redirecciones o tuberías? (Ejemplo: `" echo hola > a.txt"` o `" cat run_commands.c | grep int"`).

Ejercicio 2: Creación y Paso de Parámetros a Hilos

- **Objetivo:** Experimentar con la biblioteca `pthreads` (compilar con `-pthread`).
- **Dinámica:** Crear hilos cuya prioridad y número se definan mediante argumentos.

Requisitos del programa `hilos.c`:

- Pasar a cada hilo una estructura con: un entero (número de hilo) y un carácter (Prioritario 'P' o No prioritario 'N').
- Los hilos pares son prioritarios, los impares no.
- **¡Crucial!** Usar memoria dinámica (`malloc`) independiente para los argumentos de cada hilo. El hilo debe copiar los datos a variables locales y liberar la memoria.

Experimento:

- ¿Qué sucede si se usa una única variable de argumento compartida para todos los hilos? Explicar la razón del comportamiento.

Ejemplo hilos de clase

- Recordar el ejemplo que vimos en clase sobre la creación de hilos:
  - Adaptarlo para que `arg` no sea NULL como en el `threads_example.c`

threads_example.c

```
#include <stdio.h>
#include <pthread.h>
#define MAX_THREADS 10

void* func(void* arg) {
    printf("Thread %d \n", pthread_self());
    pthread_exit(0);
}

int main(void) {
    int j;
    pthread_attr_t attr;
    pthread_t thid[MAX_THREADS];

    pthread_attr_init(&attr);

    for(j = 0; j < MAX_THREADS; j ++)
        pthread_create(&thid[j], &attr, func, NULL);

    for(j = 0; j < MAX_THREADS; j ++)
        pthread_join(thid[j], NULL);

    return 0;
}
```

Ejercicio 3: Manejo de Señales

- **Objetivo:** Controlar el ciclo de vida de un proceso hijo mediante alarmas y señales.

Flujo del programa:

1. El proceso padre recibe por argumento el comando que ejecutará el hijo.
2. El padre crea al hijo (`fork`) y este cambia su ejecutable con `execvp`.
3. El padre configura un manejador para la señal `SIGALRM` usando `sigaction`.
4. El padre programa una alarma a los 5 segundos (`alarm(5)`).
5. Si pasan los 5 segundos, el manejador del padre envía un `SIGKILL` al hijo.
6. El padre espera al hijo (`wait`) e imprime cómo terminó (normal o por señal).

Modificación final:

- Modificar el padre para que ignore la señal `SIGINT` (Ctrl+C) y comprobar que funciona.

Esqueleto

signalsyfork.c

```
void tratar_alarma(void)
{
    kill(pid, ...);
}
int main(....) {
    struct sigaction act;
    ...
    pid = fork();
    switch(pid) {
    case -1: /* error del fork() */
        perror ("fork"); exit(-1);
    case 0:  /* proceso hijo */
        execvp(....);
        ....
    default: /* padre */
        /* establece el manejador */
        act.sa_handler = (void *)tratar_alarma;
        act.sa_flags = 0; /* ninguna acción específica */
        sigaction(SIGALRM, &act, NULL);
        alarm(5);

        wait(&status);

        if (WIFEXITED(status)) {
            printf("Finalización normal\n");
```

Ejercicio 4: Ficheros con Múltiples Procesos e Hilos

- **Objetivo:** Analizar problemas de concurrencia y carreras de diseño al escribir en un archivo común (`output.txt`).
- **Contexto:** 10 procesos (padre y 9 hijos) intentan escribir de forma concurrente 5 caracteres repetidos cada uno ("00000", "11111", ..., "99999").

El problema inicial:

- El código de partida genera salidas corruptas, desordenadas y con pérdida de datos debido a que los procesos comparten la posición del descriptor de fichero de forma descontrolada tras el `fork`.

Ejercicio 4: Tareas a Resolver

Cuestión A

- Corregir la implementación inicial para que la escritura sea concurrente y correcta (resultado final: `000001111122222...99999`).
- **Restricción:** No se debe imponer un orden de ejecución rígido ni sincronización explícita entre los procesos.

Cuestión B

- Modificar la solución para que el padre intercale su escritura entre la de cada uno de sus hijos.
- **Resultado esperado:** `0000011111000002222200000333330000...0000099999`